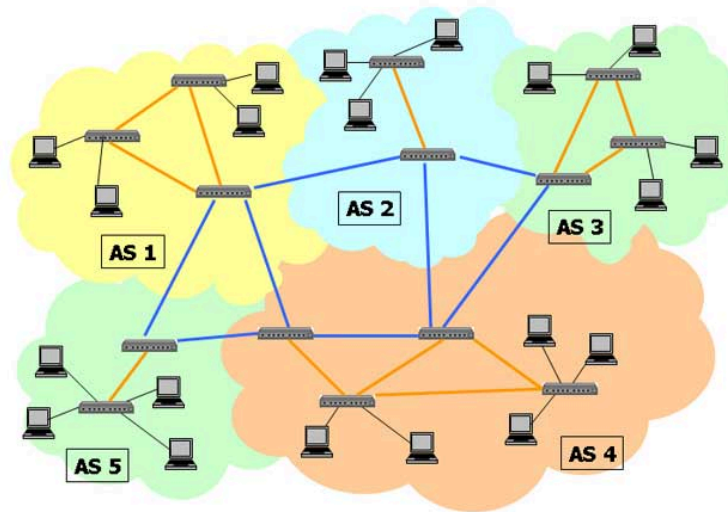


Project #2: Multi-AS Network Simulation in ns-3



In this project, you are expected to design and analyze a multi-Autonomous System (AS) network topology using the **ns-3 network simulator**. The goal is to investigate how different network configurations affect routing behavior and overall network performance.

The simulation must be implemented using **ns-3** with C++/Python (both are acceptable). And the visualization should be implemented in NetAnim.

Network Topology

You must construct a network topology consisting of **three Autonomous Systems (AS1, AS2, and AS3)**. Each Autonomous System should contain multiple routers/nodes, and communication must be established both **within each AS** and **between different AS domains**.

Inside each Autonomous System, router-to-router communication should be modeled using an **OSPF-like structure**. This means that routing decisions within an AS should be based on shortest-path behavior, where packets are forwarded through the most efficient available path according to the defined link costs or topology.

Communication between different Autonomous Systems should be modeled using a **BGP-like structure**. This means that inter-AS communication should include border routers and at least one alternative path between AS domains, allowing students to observe how traffic can be forwarded across autonomous systems and how routing behavior changes under different network conditions.

Your implementation does not need to reproduce the full real-world OSPF or BGP protocols. However, it must satisfy the following basic requirements:

- The network must include **three separate Autonomous Systems**.
- Each AS must contain routers/nodes connected internally.

- Intra-AS routing must follow an **OSPF-like shortest-path behavior**.
- Inter-AS routing must follow a **BGP-like inter-domain routing behavior**.
- At least one communication flow must pass from one AS to another.
- The simulation must be implemented in **ns-3**, using either **C++** or **Python**.
- The resulting network topology and packet flow must be visualized using **NetAnim**.

Experimental Design

You are expected to evaluate your network under different configurations.

1. Node Count

- 20 nodes
- 50 nodes
- 100 nodes

2. AS Distribution

- **Balanced**
 - Example: 33 / 33 / 34
- **Unbalanced**
 - Example: 20 / 30 / 50

So there will be 6 different configurations. (20 nodes-balanced, 20 nodes-unbalanced, 50 nodes-balanced etc.)

Performance Metrics

For each simulation scenario, you must measure:

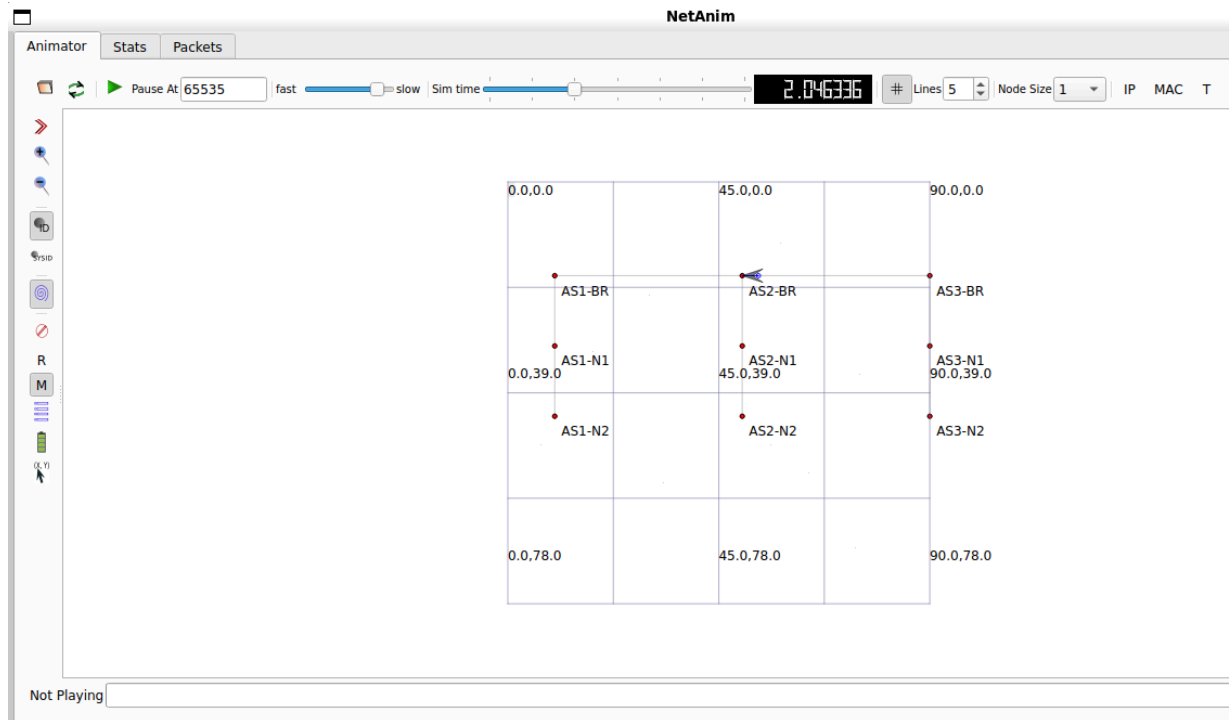
- **End-to-End Delay**
- **Throughput**
- **Packet Loss**
- **Convergence Time**

and you should save them as a table or .csv to compare the metrics for different scenarios later.

Visualization

Simulation outputs should be visualized using **NetAnim**.

The animation should clearly show the network topology, packet flow between nodes and the interaction between different AS domains.



Deliverables

You must submit:

1. Your Code: .py or .cc files.

2.Simulation Outputs: Short video recordings of simulations and .xml files (to make a NetAnim simulation, you should create .xml files with your code.)

3. Performance Results: A table or CSV file including delay, throughput, packet loss, convergence for each scenario.

4. Report: Your report must include description of your topology, Explanation of routing structure (OSPF-like & BGP-like), Comparison of different scenarios and your Interpretations of results.

Analysis Expectations

You are expected to compare different scenarios and answer questions such as:

- How does increasing node count affect performance?
- What is the impact of balanced vs. unbalanced AS distribution?
- How does the network respond to topology changes?
- Are there observable bottlenecks in routing?

Important Notes and Hints:

-You can run ns-3 in Mac/Linux but if you are using Windows, you should build it in wsl, a way to use Linux cmd in Windows, you can easily configure it in Windows terminal.

-For my implementation, C++ was easier to build, Python takes too much time and it is hard to build all dependencies. So I suggest you to use C++.

-Create .xml files with your codes, to run the simulation, your code needs to create an .xml file and you should use it in NetAnim.

-I am using Windows, since I couldn't use ns-3 without WSL, I wrote the code in cmd. So for that you should install C++/Python to your CMD.

-You are allowed to use AI tools, but plagiarism from your friends **will be calculated as 0**.

-You may benefit from ns-3 helper classes such as:

- **PointToPointHelper** for creating links between nodes
- **Ipv4AddressHelper** for assigning IP addresses
- **InternetStackHelper** for installing protocol stacks
- **Ipv4GlobalRoutingHelper** for routing configuration

These helpers can significantly simplify topology creation and routing setup.

-You can use the following links for installations:

- ns-3 can be downloaded as a compressed (ZIP or TAR) file from the official website. However, downloading alone is not sufficient; the simulator must be configured and built using the provided build system (./ns3 configure and ./ns3 build) within a Linux environment such as WSL. (<https://www.nsnam.org/releases/>)
- NetAnim can be downloaded either from the official ns-3 website or from its GitLab repository as a compressed (ZIP/TAR) file. The tool must then be built using CMake before execution. (<https://www.nsnam.org/wiki/NetAnim> , <https://gitlab.com/nsnam/netanim>)

-Feel free to ask questions!

Rubric (Total: 100 pts)

1. Network Design & Routing Logic (40 pts):

This criterion evaluates the core correctness and design quality of the network.

Correct implementation of:

- OSPF-like intra-AS routing (shortest-path behavior)
- BGP-like inter-AS routing (inter-domain communication)
- Presence of inter-AS communication flows
- Evidence of alternative routing paths or routing behavior changes

2. Simulation Implementation & Execution & Visualization (30 pts):

This criterion evaluates whether the simulation is functional and reproducible.

- Simulation runs successfully without errors
- Code correctly implements the designed topology
- Use of ns-3 components (helpers, routing, addressing, etc.) is appropriate
- Outputs (.xml, metrics) are correctly generated
- Topology and packet flow are observable

3. Experimental Design & Scenarios & Performance Evaluation (15 pts):

This criterion evaluates how well different configurations are explored.

Implementation of required scenarios:

- Node counts (20, 50, 100)
- Balanced vs. unbalanced AS distributions
- Scenarios are meaningfully constructed and comparable
- Parameters are used to reconfigure the same system, not separate hardcoded setups
- Correct measurement of delay, throughput, packet loss, convergence.
- Results are clearly organized (table / CSV)
- Metrics are consistent across scenarios

4. Analysis & Interpretation & Report (10 pts)

This criterion evaluates understanding of results.

Clear comparison between scenarios, meaningful interpretation of performance differences, routing behavior, and possible bottlenecks.